

MASTERARBEIT

Unsupervised Environment Design for Cooperative Multi-Agent Reinforcement Learning Domains

Kateryna Hamii

Entwurf vom July 4, 2026



MASTERARBEIT

Unsupervised Environment Design for Cooperative Multi-Agent Reinforcement Learning Domains

Kateryna Hamii

Aufgabensteller: Prof. Dr. Claudia Linnhoff-Popien

Betreuer: Maximilian Zorn
Philipp Altmann

Abgabetermin: 08. September 2026



Ich erkläre hiermit, dass ich die vorliegende Masterarbeit selbstständig und nur mit den angegebenen Hilfsmitteln verfasst habe. Alle Passagen, die ich aus einer Literatur oder anderen Quellen übernommen habe, sind eindeutig als Zitate mit Quellenangabe gekennzeichnet. Diese gedruckte Fassung ist ein Ausdruck der eingereichten elektronischen Fassung.

München, den 08. September 2026

.....
(*Unterschrift des Kandidaten*)

Contents

1. Introduction	3
2. Background	5
2.1. Reinforcement learning	5
2.1.0.1. The basic components of the RL model	5
2.1.0.2. The agent-environment interaction cycle	6
2.1.0.3. Formal Objective	6
2.1.0.4. Bellman Equations	7
2.1.0.5. Markov Decision Processes	8
2.1.0.6. Value Functions	8
2.1.0.7. Dynamic Programming	9
2.1.0.8. Monte Carlo Methods and Temporal-Difference Learning	9
2.1.0.9. TD Control Methods: SARSA and Q-Learning	9
2.1.0.10. Exploration and Exploitation	10
2.1.0.1. UED in multi-agent scenarios	11
2.1.1. Multi-agent reinforcement learning	13
2.1.1.1. Dec-POMDPs.	13
2.1.1.2. Partial Observability and Stochasticity	13
2.1.2. Methods for UED	13
2.1.2.1. Sparse and dense reward	13
2.2. Related works	13
3. Related Works	15
3.0.1. Curriculum Learning in Reinforcement Learning	15
3.0.1.1. (manual) curriculum design	15
3.0.1.2. Automated Curriculum Learning	15
4. Problem Formulation	17
5. Methodology	19
6. Experiments & Results	21
7. Discussion & Conclusion	23
8. Einleitung	25
8.1. Unterverzeichnisse und Dateien	25
8.1.1. main.tex	25
8.1.2. hyphenation.tex	25
8.1.3. Makefile	25
8.1.4. text	25

8.1.5.	pictures	25
8.1.6.	bibliography.bib	26
8.1.7.	mdsg.sty	26
8.2.	Beispiele	26
8.2.1.	Zitate	26
8.2.2.	Listen	26
8.2.3.	Referenz auf anderen Text	26
8.2.4.	Hoch- und tiefgestellter Text	27
8.2.5.	Tabellen	27
8.2.6.	Bilder	27
8.2.7.	Programm Code	27
8.2.8.	Fußnoten	29
8.2.9.	Formeln	29
A.	Beispiel Anhang	31
	List of Figures	33
	List of Tables	35
	Listings	37
	Bibliography	39

Abstract

Place your english abstract here.

Abstract

Unsupervised Environment Design (UED) has emerged as a promising paradigm in the single-agent reinforcement learning (RL) setting, where automatically generated curricula of progressively more complex environments lead to robust and generalizable policies.

The goal of this thesis is a systematic transfer of the UED approach to cooperative multi-agent reinforcement learning (MARL) domains. As such, the thesis shall explore the design, implementation and evaluation of environments with generalizing elements - i.e., underspecified parameters - that may be realized as various parameterized aspects of cooperation and coordination. The underspecified parameters of the environment may further include structural constraints such as layout, resource distribution, and partial observability. As an application domain, a cooperative benchmark with parameterizable layouts, for example from the Overcooked domain or similar Grid-based navigation task domains, may be considered. The dynamic aspects of the UED method, the task-design, the environment scenarios and curriculum discovery suitability will be analyzed in detail.

This thesis shall further investigate the extent to which current autocurricula from the literature — such as Prioritized Level Replay (PLR), ACCEL, SFL or MAESTRO — can be applied to parameterized multi-agent environments that require cooperation, in order to induce emergent cooperative behaviors through increasing environmental complexity. For this purpose, conventional reinforcement learning algorithms or heuristics from the literature, as well as other self-generated baselines may be used and compared. The findings aim to contribute to a deeper understanding of how open-ended environment design facilitates the emergence of cooperation in multi-agent systems.

1. Introduction

Agents need to work well with new partners even if they have never worked together before. This is called **zero-shot coordination (ZSC)**. **Overcooked** is now one of the most commonly used benchmarks for measuring the coordination abilities of AI agents and learning algorithms.

Modern approaches to reinforcement learning (RL) have made significant progress in solving complex decision-making problems, particularly in gaming environments, robotics and autonomous control systems. At the same time, a key limitation remains the poor generalisation ability of trained policies beyond the training distribution of environments. In most cases, agents demonstrate high performance in familiar scenarios but suffer a significant drop in performance when the task structure or environmental parameters change.

This problem is particularly evident in complex multi-agent systems, where the successful completion of a task depends not only on individual strategies, but also on the agents' ability to coordinate their actions. In cooperative environments, additional difficulties arise relating to the distribution of roles, partial observability, the ambiguity of optimal strategies, and the need to establish shared behavioural conventions. Under such conditions, even minor changes in the structure of the environment can lead to a significant disruption in coordination between agents.

A particular challenge is the problem of generalisation in scenarios involving parameterised or variable environments. Most standard approaches to MARL assume a fixed environment structure during training, which limits the agents' ability to adapt to new task configurations. This creates a mismatch between training conditions and real-world applications, where the environment is dynamic and partially unknown.

In order to address these limitations, the Unsupervised Environment Design (UED) approach was proposed, which treats the generation of environments itself as part of the learning process. Instead of using a predefined set of tasks, UED automatically generates environments that adapt to the agent's current level and stimulate its learning by progressively increasing the difficulty or adding variety to the scenarios. In the single-agent case, this approach has demonstrated the ability to significantly improve policy generalisation.

In order to address these limitations, the Unsupervised Environment Design (UED) approach was proposed, which treats the generation of environments itself as part of the learning process. Instead of using a predefined set of tasks, UED automatically generates environments that adapt to the agent's current level and stimulate its learning by progressively increasing the difficulty or adding variety to the scenarios. Unsupervised Environment Design approaches demonstrate improved generalisation to unseen environments compared to training on a fixed task distribution. [DJV⁺20]

At the same time, the application of UED in cooperative multi-agent systems remains

1. Introduction

under-researched. The main challenge lies in the fact that, in a MARL environment, it is necessary to consider both the individual learning of agents and the process of forming coordination strategies between them simultaneously. This complicates the design of the training curriculum, as changes in the environment can affect not only the complexity of the task but also the stability of interactions between agents.

Furthermore, modern methods for the automatic generation of training scenarios, such as Prioritized Level Replay (PLR), ACCEL, Sampling for Learnability (SFL) and MAE-STRO, have primarily been developed for single-agent or limited-cooperation settings. Their effectiveness in tasks where the emergence of complex coordination strategies in multi-agent environments is critical remains under-researched.

In this context, parameterised cooperative environments are of particular interest, in which the structural characteristics of the task—such as the configuration of the space, the location of resources and the level of observability—can change during the learning process. Such environments allow for the systematic investigation of how changes in task structure affect the emergence of cooperation and agents’ ability to generalise. The aim of this master’s thesis is to investigate the applicability of Unsupervised Environment Design approaches in cooperative multi-agent environments with a parameterised structure, as well as to analyse their impact on the formation of generalisable cooperative behaviour in agents.

Structure

The thesis consists of an introduction, a theoretical review, a description of the methodology, an experimental section, an analysis of the results, and conclusions. The first chapter examines the fundamentals of reinforcement learning and multi-agent systems, including the formalisation of the problem and the issue of partial observability. The second chapter is devoted to methods of automatic environment generation and approaches to constructing training environments. The third chapter describes the proposed formulation of the problem and a parameterised environment for cooperation. The fourth chapter includes an experimental evaluation of the methods and an analysis of the results. The paper concludes with conclusions and possible directions for further research.

2. Background

2.1. Reinforcement learning

Reinforcement learning (RL) refers to a subfield of machine learning in which an agent learns through interaction with an environment. Sutton and Barto (2018) [SB18] define this paradigm as a process in which a sequence of decisions is made to maximise a cumulative reward signal. In contrast to supervised learning, where an external teacher specifies correct target values, in RL the agent receives only a scalar reward signal as feedback on its actions. The agent must therefore independently determine which actions yield the highest return in the long term. The authors highlight three key characteristics of this approach: Firstly, it is a closed-loop system in which the agent's actions influence its future inputs. Secondly, the agent receives no explicit instructions as to which actions are optimal. Thirdly, the consequences of actions can extend over extended periods of time.

2.1.0.1. The basic components of the RL model

The fundamental architecture of RL is based on the conceptual distinction and closed-loop interaction between two key entities: the Agent and the Environment. According to Sutton and Barto's [SB18] fundamental definition, the basic RL setting consists of the following elements:

- **Agent:** A control subsystem (controller) capable of perceiving information about the state of the external environment, interpreting it, making decisions regarding the choice of actions, and adapting its own parameters (learning) during operation.
- **Environment:** A dynamic system that lies outside the Agent's immediate internal control but is subject to its influence. The environment encompasses all physical or virtual constraints, transition laws between states, and feedback generation mechanisms.
- **State** ($s \in \mathcal{S}$): a complete or partial informational representation of the environment's configuration at a specific discrete point in time t . The state space $\mathcal{S}$ can be either finite (discrete) or infinite-dimensional (continuous). Ideally, a state $s_t \in \mathcal{S}$ satisfies the Markov property, i.e., it contains all relevant retrospective information necessary for predicting the future response of the system.
- **Action** ($a \in \mathcal{A}(s)$): A control signal or an act by which the Agent exerts a specific influence on the Environment. The set of available actions $\mathcal{A}(s)$ is a function of the current state, reflecting the dynamic constraints imposed on the Agent's behaviour in specific system configurations.
- **Reward** ($r \in \mathcal{R} \subset \mathbb{R}$): A scalar numerical feedback signal R_t generated by the Environment after each step of the Agent. The physical meaning of the reward lies in the local evaluation of the quality of the transition $(S_{t-1}, A_{t-1}) \rightarrow S_t$. The reward mechanism is the sole carrier of information about the global control objective.

2. Background

- **Strategy / Policy** (Policy, π): A specification of the Agent’s behaviour that defines the mapping from the state space to the action space. In the general stochastic case, it is formalised as a conditional probability distribution:

$$\pi(a|s) = \mathbb{P}(A_t = a \mid S_t = s), \quad \sum_{a \in \mathcal{A}(s)} \pi(a|s) = 1, \quad \forall s \in \mathcal{S}.$$

- **Episode**: A sequence of interactions that naturally breaks down into time intervals, each ending in a specific state—the terminal state. Problems with this structure are called episodic.
- **Return** (G_t): A function of the sequence of rewards that the agent seeks to maximise. In the most general form, taking discounting into account, the return is defined as:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

- **Discount factor** (γ): A parameter in the interval $0 \leq \gamma \leq 1$ that determines the present value of future rewards. When $\gamma < 1$, the sum in the revenue formula is finite if the reward sequence is bounded.

2.1.0.2. The agent-environment interaction cycle

The agent’s interaction with the environment occurs continuously through a sequence of discrete time steps $t = 0, 1, 2, 3, \dots$. This iterative process follows the pattern shown in Figure 2.1: Agent $\rightarrow$ Action $\rightarrow$ Environment $\rightarrow$ Reward + Next state. At each time step t , the agent perceives the state of the environment S_t and, based on this, performs action A_t . At the next time step, as a partial consequence of this action, the agent receives a numerical reward R_{t+1} from the environment and finds itself in a new state S_{t+1} . This cycle forms a continuous trajectory of the agent’s experience.

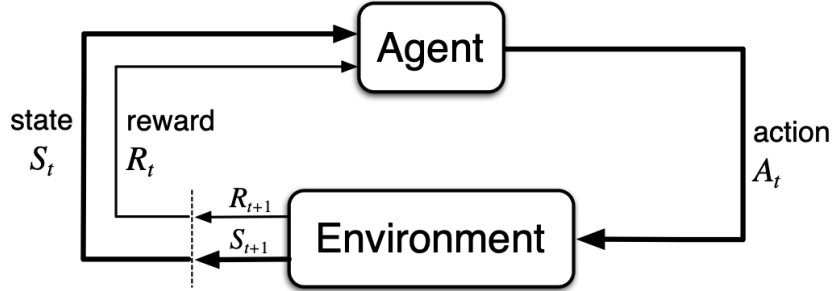


Figure 2.1.: Agent-environment interaction cycle

2.1.0.3. Formal Objective

The entire paradigm of reinforcement learning is based on the so-called “*reward hypothesis*”: any goal or objective can be formalised as the maximisation of the expected value of the cumulative sum of the received scalar reward signal. To formalise this long-term

goal, the concept of return (G_t) is introduced. For continuous-time problems, the return is defined as the sum of discounted future rewards:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (2.1)$$

To assess how beneficial it is for an agent to be in a particular state s when following a given policy π , the state-value function ($v_\pi(s)$) is used. It is defined as the expected return starting from state s :

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right] \quad (2.2)$$

Thus, the formal objective of an agent in reinforcement learning is to find the optimal policy (π^*), whose expected reward is greater than or equal to that of any other policy for every state without exception. Mathematically, this means achieving the optimal state value function ($v^*(s)$), defined as:

$$v^*(s) = \max_{\pi} v_\pi(s) \quad (2.3)$$

which maximises the expected reward for every state of the environment.

Thus, rather than globally optimising an abstract function, RL algorithms seek behaviour that will maximise the expected return ($v^*(s)$) for each local state in which the agent may find itself.

2.1.0.4. Bellman Equations

A fundamental property of value functions is their recursive structure, which is expressed by the Bellman equation. For the state-value function, the following relationship holds as formulated by Sutton and Barto (2018):

$$v_\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')].$$

This equation expresses that the value of a state equals the expected discounted value of successor states plus the expected immediate reward. It forms the foundation of all dynamic programming and reinforcement learning algorithms.

For the optimal state-value function $v^*(s)$, the Bellman optimality equation is given by:

$$v^*(s) = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v^*(s')].$$

Similarly, for the optimal action-value function $q^*(s, a)$, it holds that:

$$q^*(s, a) = \sum_{s', r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q^*(s', a') \right].$$

Sutton and Barto [SB18] show that any policy that selects, in every state, only the action with the highest q^* -value (i.e., greedy with respect to v^* or q^*) constitutes an optimal policy π^* .

2. Background

2.1.0.5. Markov Decision Processes

The mathematical foundation of Reinforcement Learning is the Markov Decision Process (MDP). Following the standard work of Sutton and Barto [SB18], a finite MDP is defined by the tuple (S, A, p, r, γ) , where S denotes the finite set of states, A the finite set of actions, p the transition dynamics, r the reward function, and $\gamma \in [0, 1]$ the discount factor.

At each discrete time step t , the agent is in a state $S_t \in S$ and selects an action $A_t \in A(S_t)$. The environment then transitions to a new state S_{t+1} , and the agent receives a numerical reward R_{t+1} .

The dynamics of the MDP are fully described by the joint transition and reward probability:

$$p(s', r \mid s, a) = \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\}.$$

This formulation assumes the Markov property. As explained by Sutton and Barto (2018) [SB18], the probability of a state transition depends solely on the current state and the current action, rather than on the entire history. Formally, this means that the complete dynamics are identical to the conditional probability given the full history

$$S_0, A_0, R_1, \dots, S_t, A_t.$$

2.1.0.6. Value Functions

Almost all reinforcement learning algorithms involve estimating value functions—functions of states (or state–action pairs) that assess how good it is for an agent to be in a given state or how good it is to take a certain action in that state. The notion of “how good” is defined in terms of future rewards that the agent is expected to receive, or more precisely, in terms of the expected return. Since expected rewards depend on the actions chosen by the agent, value functions are always defined relative to a particular policy π .

In reinforcement learning theory, two main types of value functions are distinguished:

1. ****State-value function**** $v_\pi(s)$ The value of a state s under a policy π , denoted $v_\pi(s)$, is the expected return that an agent obtains when starting from state s and thereafter following policy π . Formally, for Markov Decision Processes (MDPs), this function is defined as follows:

$$v_\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right]$$

This mathematical construct is called the state-value function for policy π . By definition, the value of any terminal state is always equal to zero.

2. ****Action-value function**** $q_\pi(s, a)$ Similarly, the value of taking a specific action a in state s under policy π , denoted $q_\pi(s, a)$, is defined. It is the expected return when the agent starts from state s , takes action a , and then continues to follow policy π . Mathematically, it is written as:

$$q_\pi(s, a) = \mathbb{E}_\pi [G_t \mid S_t = s, A_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right]$$

This function is called the action-value function for policy π .

2.1.0.7. Dynamic Programming

Dynamic Programming (DP) comprises a class of algorithms that can compute optimal policies given a known environment model. The framework of Generalized Policy Iteration (GPI), as described by Sutton and Barto (2018), alternates between two central procedures: policy evaluation and policy improvement.

During policy evaluation, the Bellman equation is iteratively solved to approximate v_π for a given policy π . During policy improvement, a better policy is constructed by selecting, in each state, the action that maximizes the value function. The repeated application of these two steps converges to the optimal policy.

However, DP methods assume full knowledge of the environment dynamics $p(s', r | s, a)$ and become computationally expensive in large state spaces.

2.1.0.8. Monte Carlo Methods and Temporal-Difference Learning

In contrast to dynamic programming methods, Monte Carlo (MC) methods do not require a model of the environment. They estimate value functions based on complete episodes by using the actual return G_t as a target:

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)].$$

Temporal-Difference (TD) learning, extensively discussed by Sutton and Barto (2018), combines properties of both approaches. Like Monte Carlo methods, TD learning learns directly from experience without requiring a model; like dynamic programming, it updates estimates based on other learned estimates (bootstrapping).

The basic TD(0) update rule is given by:

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)].$$

The term

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

is known as the temporal-difference error (TD error) and represents a central concept in reinforcement learning.

2.1.0.9. TD Control Methods: SARSA and Q-Learning

For the control problem, i.e., finding an optimal policy, temporal-difference methods are applied to action-value functions. Two central algorithms in this context are SARSA (on-policy) and Q-learning (off-policy).

SARSA updates the action-value function along the trajectory of the policy that is actually being followed:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)].$$

The name SARSA is derived, as described by the authors, from the quintuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ used in each update.

In contrast, Q-learning directly approximates the optimal action-value function q^* ,

2. Background

independent of the policy being followed:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right].$$

The key difference is that Q-learning uses the maximum action value over all possible actions in the next state ($\max_a$), whereas SARSA uses the value of the actually executed next action. Under mild conditions, Q-learning converges to q^* with probability 1.

2.1.0.10. Exploration and Exploitation

A central dilemma in reinforcement learning is the trade-off between exploration (trying new actions) and exploitation (selecting actions known to yield high rewards). Sutton and Barto (2018) discuss several approaches to address this problem.

A widely used strategy is the ε -greedy policy, where the agent selects the currently best-known action with probability $1 - \varepsilon$ and a random action with probability ε .

Other approaches described by the authors include the Upper Confidence Bound (UCB) method and softmax-based action selection strategies.

The research of robust and cooperative agents in multi-agent systems requires a synthesis of different research fields: Unsupervised Environment Design (UED), multi-agent reinforcement learning (MARL) and automated generation of learning plans.

Unsupervised Environment Design

Unsupervised environment design was introduced as an approach to automatically generate learning environments that enable agents to develop more robust behaviour. One of the seminal works is by [BKM⁺19], who, in ‘*Emergent Tool Use from Multi-Agent Autocurricula*’, demonstrate that complex skills can emerge through adversarial environment generation. In this process, a ‘teacher’ generates environments, while a ‘student’ attempts to solve them.

Building on this, the UED paradigm has been systematised, for example through work such as PAIRED (Protagonist Antagonist Induced Regret Environment Design), which specifically generates challenging but solvable environments ([DJV⁺20]). This work demonstrates that automatically generated curricula enable greater generalisation than static training environments. However, much of the research to date has focused on single-agent scenarios, meaning that its applicability to cooperative multi-agent systems has only been investigated to a limited extent so far.

UED is often described as a two-player game between an level-solving agent and a level-generating adversary [MLB⁺25]. A key metric for level selection is regret [DJV⁺20]—the difference between the potentially optimal performance on a level and the actual performance of the current agent. The best-known methods can be divided into two categories:

- Replay-based UED methods: Rather than having a neural network generate environments from scratch each time, algorithms such as Prioritised Level Replay (PLR) [JGR20] rely on a buffer of previously seen levels. PLR evaluates randomly generated levels based on their learning potential (e.g. approximated by the Positive Value Loss) and trains the agent with a priority on those environments that exhibit

the highest approximated regret, leading to significantly more robust policies in grid-based navigation tasks [PHJD⁺23].

- Evolutionary and mutation-based UED methods: The ACCEL algorithm (Adversarially Compounding Complexity by Editing Levels) [PHJD⁺23] extends PLR with a mutation operator. It makes small adjustments to levels that already exhibit high regret, thereby continuously pushing the agent to the limits of its capabilities, which causes the complexity of the environments to increase steadily
- Learnability-based approaches: Recent research shows that regret approximations are often inaccurate in practice and do not always correlate with the true learning potential. Sampling For Learnability (SFL) addresses this by measuring the learnability of a level based on the variance of the success rate ($p(1 - p)$) [RBW⁺24]. SFL consequently prioritises environments that the agent can solve ‘sometimes, but not always’, leading to significantly more robust policies in grid-based navigation tasks.

2.1.0.1. UED in multi-agent scenarios

While UED has been extensively researched in a single-agent context, its application to multi-agent systems is still in its early stages [DJV⁺20, JGR20, PHJD⁺23]. In multi-agent domains, there is the additional challenge that the complexity of a task depends not only on the static environment, but is also significantly determined by the strategies of the other players (co-players) [SKD⁺23]

Previous approaches to generating auto-curricula in MARL have primarily focused on the evolution of opponent policies in competitive games, such as through fictitious self-play (FSP) or population-based training [SKD⁺23]. The recently introduced MAESTRO (Multi-Agent Environment Design Strategist for Open-Ended Learning) algorithm is one of the first approaches to transfer UED to competitive MARL environments (two-player zero-sum games) [SKD⁺23]. MAESTRO combines PLR with population-based learning to generate a common curriculum across environments and opponent policies, thereby making agents more robust to both environmental changes and unknown opponents [SKD⁺23]

However, in the cooperative setting that is the focus of this work, there are hardly any comparable approaches [RBPB25]. Much of the existing research on multi-agent UED either does not address fully cooperative settings, focuses on rudimentary design spaces, or investigates pathfinding without direct agent interaction [RBPB25]. Recent initiatives such as the Overcooked Generalisation Challenge [RBPB25] or the Craftax-MA [?] benchmark clearly demonstrate that classical MARL algorithms perform significantly worse in procedurally generated, open-ended cooperative tasks with long time horizons and heterogeneous requirements [OMRF25]. Early frameworks in the ‘Overcooked’ domain have already defined Dec-UPOMDPs (Underspecified POMDPs with Shared Rewards), in which layout, object distributions and starting positions are parameterised to enable zero-shot coordination across different environments [RBPB25]

To overcome the poor generalisation ability of RL agents, they must be trained across a wide variety of environments. A classic approach to this is domain randomisation (DR) [TFR⁺17], in which environment parameters are randomly sampled. However, as DR often generates unsolvable or trivial environments [DJV⁺20], Unsupervised Environment Design (UED) was introduced as a scalable approach, in which a curriculum is adaptively

2. Background

adjusted to the agent’s current capabilities [RBPB25]

Multi-Agent Reinforcement Learning and Cooperation

In the field of multi-agent reinforcement learning, the emergence of cooperative behaviour presents a key challenge. Traditional approaches such as Independent Q-Learning [Tan93] suffer from non-stationarity, as the policies of other agents change during training. Advanced methods such as QMIX [RSdW⁺18], VDN (Value Decomposition Networks) [SLG⁺17], and MADDPG [LWT⁺17] address this problem through centralised training mechanisms with decentralised execution. For cooperative scenarios, the Overcooked¹ domain [CSH⁺19] was established in particular, in which agents must coordinate tasks together. This environment allows for a controlled investigation of cooperation, communication and coordination strategies.

Solving complex coordination problems involving multiple agents is a key area of research in artificial intelligence. The fully cooperative MARL problem is often formally modelled as a decentralised partially observable Markov decision process (Dec-POMDP) [OMRF25], in which all agents act on the basis of their local observations but optimise a shared, scalar team reward signal [dWGM⁺20]

A straightforward approach is Independent Learning (IL), in which each agent treats its environment as stationary and regards the other agents merely as part of that environment [dWGM⁺20]. However, as this leads to learning instabilities, the Centralised Training with Decentralised Execution (CTDE) [LWT⁺17] paradigm has become established. Here, training is carried out centrally in a simulation where global state information is available for learning a common value function (critic), while the execution of the learned policies is strictly decentralised and based on local observations [SKS⁺19]

Although MARL algorithms such as QMIX [RSdW⁺18] or COMA [FFA⁺24] achieve high performance levels in standard benchmarks (e.g. SMAC [SRdW⁺19]), they typically train agents using so-called self-play (SP) [GEK17]. As a result, agents are heavily reliant on the conventions established during training. When these agents have to interact in new environments or with unknown partners (a problem known as zero-shot coordination or ZSC [HLPF21]), they often fail to generalise effectively.

Autocurricula and Level-Generation

Another relevant area of research is automatic curricula that adaptively select training data or environments. Key approaches include: Prioritized Level Replay (PLR) ([JGR20]) which selects training levels based on learning progress and difficulty; ACCEL (Adaptive Curriculum via Contrastive Learning): Uses contrastive learning to generate meaningful training progressions. SFL (Sampling For Learnability) is a simple approach that directly chooses levels that optimise learnability alongside randomly generated levels. [RBW⁺24]. MAESTRO (Open-Ended Learning Framework) [SKD⁺23] Combines evolution and curriculum learning for the open-ended generation of tasks.

These methods demonstrate that dynamic curricula are crucial for efficient learning. Nevertheless, they have predominantly been evaluated in single-agent contexts.

¹https://github.com/HumanCompatibleAI/overcooked_ai

Combination of UED and MARL

The integration of UED into cooperative multi-agent scenarios has so far been insufficiently researched. Although UED has been successfully deployed in competitive environments, cooperative settings give rise to additional requirements, particularly with regard to coordination and the joint achievement of goals.

Parameterised environments — such as varying layouts, resource distributions or partial observability — offer the possibility of specifically modulating cooperation requirements. In this context, the question arises as to whether principles such as learnability, as defined in SFL, are also transferable to multi-agent scenarios, for example through shared success metrics or coordination-dependent levels of difficulty

2.1.1. Multi-agent reinforcement learning

2.1.1.1. Dec-POMDPs.

Dec-POMDPs. A Dec-POMDP is defined as a tuple $(n, S, A, T, O, O, R, \gamma)$ [OSV08]. Here, $n \in \mathbb{N}$ denotes the number of agents, S the state space, A is the action space for each agent, and O represents an agent’s observation space. The transition probability function $T : S \times A^n \rightarrow \Delta(S)$ dictates the evolution of the environment, the observation function $O : S \times A^n \rightarrow \Delta(O^n)$ determines what agents can perceive, and $R : S \times A^n \rightarrow \mathbb{R}$ is the reward function. Finally, $\gamma \in [0, 1)$ is the discount factor. At every timestep t , each agent i selects an action $a_t^i \in A$. The environment then transitions from state $s_t \in S$ to $s_{t+1} \in S$ according to $T(s_t, a)$ with probability $P(s_{t+1} \mid s_t, a_t)$, where $a_t = (a_t^1, \dots, a_t^n)$ represents the joint action. A reward $r_t \in \mathbb{R}$ is drawn based on $R(s_t, a_t)$, and each agent receives an observation o_{t+1}^i , with the joint observation $o_{t+1} = (o_{t+1}^1, \dots, o_{t+1}^n)$ sampled from $O(s_{t+1}, a_t)$. The primary objective is to learn policies $\pi^i : (O^i \times A^i)^* \rightarrow \Delta(A^i)$ for each agent i , which maximises the expected cumulative discounted reward $\mathbb{E} \left[\sum_{t=0}^T \gamma^t r_t \right]$.

2.1.1.2. Partial Observability and Stochasticity

2.1.2. Methods for UED

- Constant: domain randomization [?]
- Regret-Based: [DJV⁺20] minimax regret, [PHJD⁺23] PLR, [JGR20]

2.1.2.1. Sparse and dense reward

2.2. Related works

3. Related Works

3.0.1. Curriculum Learning in Reinforcement Learning

3.0.1.1. (manual) curriculum design

The concept of curriculum learning originated in the early 1990s and is based on the idea that learning can be improved by presenting training examples in a meaningful order. One of the earliest demonstrations was provided by [Elm93] (1993), who showed that neural networks achieved better language learning performance when training started with simple examples before gradually introducing more complex ones. Similar findings were reported by Rohde and Plaut (1999) ([RP99]), who further investigated the role of incremental learning in language acquisition. Around the same time, Sanger (1994) ([San94]) applied the same principle to robotic control, demonstrating that gradually increasing task difficulty improved the learning of neural network controllers for robot manipulators.

The concept was later formalized by Bengio et al. (2009) ([BLCW09]), who introduced curriculum learning as a general machine learning paradigm in which training samples are presented in an order of increasing difficulty. Their work argued that, similar to human education, machine learning algorithms can benefit from beginning with simpler tasks and progressively tackling more challenging ones. Across these studies, a common conclusion emerged: organizing training experiences from easy to difficult can accelerate convergence during training while also improving the final performance of the learned model.

These principles naturally extend to reinforcement learning, where manually designing an appropriate curriculum is often impractical due to the complexity and diversity of possible environments. This challenge motivated the development of Automated Curriculum Learning (ACL), which aims to generate curricula automatically by adapting the difficulty of training tasks to the agent’s current learning progress ([PCW⁺20]; [NPL⁺20]).

3.0.1.2. Automated Curriculum Learning

Automated Curriculum Learning (ACL) is a research area within reinforcement learning (RL) that aims to improve an agent’s learning process by automatically adjusting the difficulty of training tasks according to the agent’s current capabilities. Rather than exposing the agent to tasks of fixed complexity, ACL methods generate a sequence of increasingly challenging tasks that facilitate more efficient learning ([PCW⁺20], [NPL⁺20]). This idea is inspired by the educational concept of the Zone of Proximal Development proposed by Vygotsky, which suggests that learning is most effective when tasks are neither too easy nor too difficult ([V⁺11]).

The primary objective of many ACL approaches is to maximize learning progress by selecting tasks that match the agent’s current skill level ([FHGA18]). Depending on the application, automated curricula can pursue different goals. Some methods focus

3. Related Works

on accelerating learning and improving performance on a predefined set of target tasks ([GDG⁺24]), while others aim to develop agents that generalize robustly to previously unseen environments ([DJV⁺20], [JDPH⁺21]).

A prominent research direction pursuing the latter objective is Unsupervised Environment Design (UED). Instead of relying on manually designed training scenarios, UED algorithms automatically generate diverse environments that challenge the learning agent and encourage the development of more robust policies ([DJV⁺20]). In these methods, an environment generator modifies characteristics of the environment, including transition dynamics, initial states, observation spaces, reward functions, or goal configurations, to create a diverse set of training tasks ([FHGA18]; [DJV⁺20]). Each specific environment configuration is commonly referred to as a level ([DJV⁺20]).

Overall, ACL provides a general framework for adapting training difficulty throughout the learning process, while UED represents a specific class of ACL methods that automatically generate increasingly informative and diverse environments to improve generalization and robustness.

4. Problem Formulation

Extend the UPOMDP/UPOSG formalisms to cooperative settings. Define what "underspecified parameters" mean in cooperative domains (layout, resource placement, partial observability, agent capabilities).

- technical specification, actual mathematical definitions of problem: – Cooperative Underspecified Dec-POMDP (shared reward, joint value, Underspecified Parameter Space)

5. Methodology

This chapter should define how to optimize (the concrete algorithms, their adaptation, the training pipeline) — **the method**

Applying UED Autocurricula to Cooperative MARL

Adapt PLR, ACCEL, and potentially MAESTRO-like mechanisms to cooperative settings. Define regret in cooperative contexts and discuss how the curriculum is shaped.

- Overview of the Training Pipeline
- Adapting algos to Cooperative MARL
- Design Decision: Co-Player Variation
- Baselines and Evaluation Protocol

(collects training params and metadata: steps, trajectories, collisions usw)

6. Experiments & Results

Comparisson of Baselines, metrics, training protocol, zero-shot generalization evaluation, emergent cooperation analysis.

7. Discussion & Conclusion

This chapter interprets what it means (results and findings) with respect to research questions from Chapter 1

Interpret findings, limitations, and future work.

8. Einleitung

Dies ist der $\text{\LaTeX}$ Rahmen zur Bearbeitung von Bachelor-, Master-, Projekt- und Diplomarbeiten. Alle relevanten Dateien befinden sich im Verzeichnis `text`.

8.1. Unterverzeichnisse und Dateien

Das Verzeichnis `text` beinhaltet weitere Unterverzeichnisse und Dateien, die den Rahmen charakterisieren.

8.1.1. `main.tex`

Diese Datei stellt die zentrale Konfigurationsdatei für den Rahmen dar. Unter anderem müssen hier Informationen über die Aufgabensteller, Betreuer, die Art der Arbeit sowie deren Title eingestellt werden. Hier können auch weitere Pakete eingebunden werden. Die Datei ist dokumentiert und sollte selbsterklärend sein.

8.1.2. `hyphenation.tex`

Manche Wörter werden von $\text{\LaTeX}$ nicht (ordentlich) getrennt. Diese können in dieser Datei mit deren Trennungsstellen hinzugefügt werden.

8.1.3. `Makefile`

Um das Dokument zu erstellen muss man den Aufruf `make all` tätigen. Dabei werden einige temporäre Dateien erstellt sowie die Datei `main.pdf` die das entsprechende Dokument enthält. Mit dem Aufruf `make clean` werden alle temporären Dateien sowie die Datei `main.pdf` gelöscht. Sie können die Datei `Makefile` ihren Anforderungen entsprechend erweitern.

8.1.4. `text`

Es bietet sich an für verschiedene Kapitel eigene Quelldateien zu pflegen. Diese sollten Sie alle im Ordner `text` ablegen. Wie ein Kapitel eingebunden wird, kann man aus dem Beispiel in der Datei `main.tex` ablesen. Das Verzeichnis `text` beinhaltet zudem die Datei `abstract.tex`. In diese Datei soll eine kurze Zusammenfassung (ca. eine halbe Seite) der Arbeit eingetragen werden. Die Datei `appendix.tex` kann verwendet werden um einen Anhang zu generieren.

8.1.5. `pictures`

Hier müssen Sie alle Grafiken ablegen, die Sie in ihrem Dokument einbinden wollen. Es sind nur die Formate PDF, PNG und JPEG erlaubt (GIF ist möglich, wird aber nicht empfohlen).

8.1.6. bibliography.bib

In diese Datei müssen alle Referenzen eingetragen werden, die innerhalb ihrer Arbeit zitiert werden. Verwenden sie zur Verwaltung ihrer Referenzen einen geeigneten Editor z.B. *JabRef* (<http://jabref.sourceforge.net/>).

8.1.7. mdsg.sty

Hierbei handelt es sich um das Stylefile, das das Erscheinungsbild des Dokuments lenkt. In dieser Datei sollten in der Regel keine Veränderungen notwendig sein.

8.2. Beispiele

Es gibt eine Unmenge an L^AT_EX Tutorials und Dokumentationen, die guten Einstieg in das Arbeiten mit L^AT_EX ermöglichen. Im Folgenden werden aber ein paar undokumentierte Minimalbeispiele gegeben, die den direkten Einstieg ermöglichen. Betrachten sie den Quelltext, um die Beispiele nachzuvollziehen.

8.2.1. Zitate

Wir zitieren hier eine Quelle von James Aspnes et al [AS07], die in der Datei `bibliography.bib` steht.

8.2.2. Listen

Es gibt verschiedene Möglichkeiten Listen zu erstellen, z.B. ohne Nummerierung...

- Das ist der erste Punkt,
 - das der erste Unterpunkt,
 - das der zweite Unterpunkt,
- das der zweite, und
- das der dritte Punkt.

...oder mit Nummerierung...

1. Das ist der erste Punkt,
 - a) das der erste Unterpunkt,
 - b) das der zweite Unterpunkt,
2. das der zweite, und
3. das der dritte Punkt.

8.2.3. Referenz auf anderen Text

Es ist auch möglich auf andere Stellen im Text z.B. Kapitel 8.1.1 zu verweisen.

8.2.4. Hoch- und tiefgestellter Text

Man kann Text tiefstellen indem man `\textsubscript` verwendet, z.B. ergibt

`text\textsubscript{tiefgestellt}`

den Text $\text{text}_{\text{tiefgestellt}}$. Das selbe funktioniert mit `\textsuperscript` verwendet, z.B. ergibt

`text\textsuperscript{hochgestellt}`

$\text{text}^{\text{hochgestellt}}$

8.2.5. Tabellen

Es gibt schöne Möglichkeiten Tabellen einzubinden wie z.B. Tabelle 8.1.

Parameter	Value	(Unit)	Available for Chord
Query timeout	10	seconds	✓
Republish timeout	300	seconds	✓
Stabilize timeout	5	seconds	✓
Fix fingers timeout	30	seconds	✓
Message timeout	1	second	✓
Connect timeout	10	seconds	✓
Ping superpeer timeout	5	seconds	×
Cost-Optimality estimation timeout	20	seconds	×
Significance for change in number of superpeers	10	percent	×
Significance for change in estimations	10	percent	×
Number of permanent superpeers	32	nodes	×
Mean number of peers	1000	nodes	✓
Mean number of lookups per hour	60	queries	✓
Mean number of shared InfoProfiles per node	20		✓
Identifier space	16	bits	✓
Direct insertion acknowledgment	true	bool	×
Direct query responses	true	bool	×
Force query resolution	true	bool	✓

Table 8.1.: Common simulation parameter settings.

8.2.6. Bilder

Man kann sehr einfach Bilder einbinden so wie z.B. in Abbildung 8.1. Es lassen sich auch mehrere Bilder nebeneinander platzieren wie z.B. in Abbildung 8.2 zu sehen ist.

8.2.7. Programm Code

Eine elegante Möglichkeit, Programmtext einzubinden, lässt sich mit dem `listings`-Paket erreichen. Das `HelloWorld` Programm aus Listing 8.1 hat in Zeile 3 übrigens einen Programmierfehler.

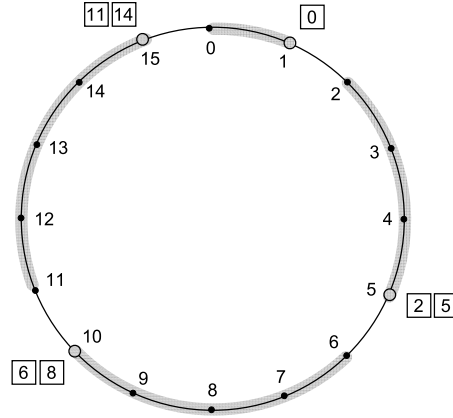


Figure 8.1.: Example of a 4-bit Chord identifier circle. The responsibility ranges for each peer are accentuated in light gray

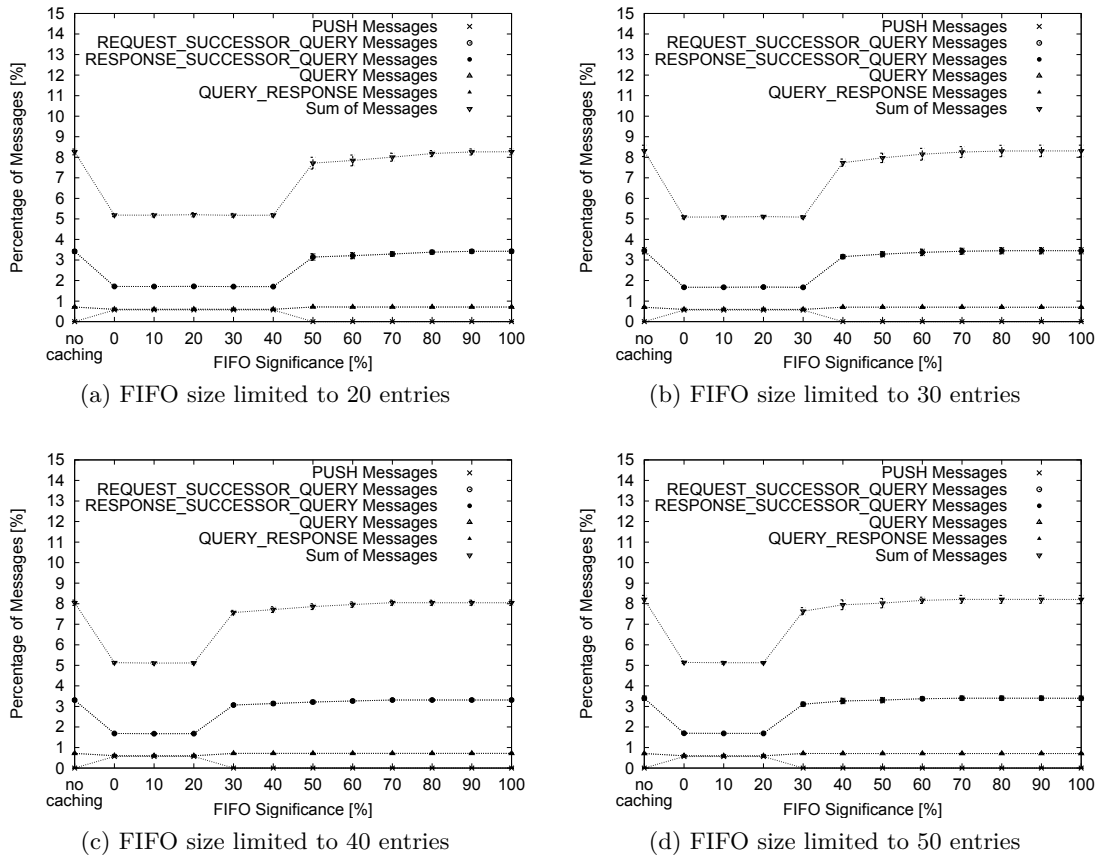


Figure 8.2.: Observed message fractions and 95% confidence intervals for Chord without the influence of churn. The FIFO capacity varies from 20 (8.2a) – 50 (8.2d) entries (decadic steps).

Listing 8.1: Hello World

```

1 public class HelloWorld {
    public static void main(String [] args) {
3         System.out.println("Hello , World");
    }
5 }

```

8.2.8. Fußnoten

Wenn man auf Google ¹ verweisen will, bietet sich statt einer gesonderten Referenz auch einfach eine Fußnote an.

8.2.9. Formeln

Man kann mit $\text{\LaTeX}$ sehr schön Formeln erzeugen:

$$L_P(k) = R_P^{orig}(k) + \sum_{i=0}^n 2 * R_P^i(k)$$

¹<http://www.google.com>

A. Beispiel Anhang

```
/*
 * This code serves the initialization of an auxiliary probability array.
 * The array holds at each position a pre-calculated probability for the index
 * of that position. The probability reflects the Zipf-distribution for the
 * corresponding indexes
 */
Set zipfExponent to 1.4
Set sum to 0
Set maxInteger to 65535
FOR i = 0 to maxInteger
  probArray[i] = 1/pow(i + 1, zipfExponent)
  Set sum = sum + probArray[i]
END FOR
FOR i = 0 to maxInteger
  Set probArray[i] = probArray[i]/sum
END FOR
FOR i = 1 to maxInteger
  Set probArray[i] = probArray[i] + probArray[i] = probArray[i-1]
END FOR

/*
 * This code gets called in case a Zipf-distributed number is required. It
 * iterates over the probability array until the chosen random number v
 * is less than the value stored at the current array position i. The value of
 * the array position will be returned as the calculated Zipf-distributed
 * number
 */
Set v to a random number between 0 and 1
FOR i = 0 to maxInteger
  IF v < probArray[i] THEN
    RETURN i
  END IF
END FOR
RETURN 0
```


List of Figures

2.1. Agent-environment interaction cycle	6
8.1. Example of a 4-bit Chord identifier circle	28
8.2. Observed message fractions and 95% confidence intervals for Chord	28

List of Tables

8.1. Simulation parameter settings	27
--	----

Listings

8.1. Hello World	29
----------------------------	----

Bibliography

- [AS07] James Aspnes and Gauri Shah. Skip graphs. *ACM Trans. Algorithms*, 3(4):37, 2007.
- [BKM⁺19] Bowen Baker, Ingmar Kanitscheider, Todor Markov, Yi Wu, Glenn Powell, Bob McGrew, and Igor Mordatch. Emergent tool use from multi-agent autocurricula, 09 2019.
- [BLCW09] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *Proceedings of the 26th annual international conference on machine learning*, pages 41–48, 2009.
- [CSH⁺19] Micah Carroll, Rohin Shah, Mark K. Ho, Thomas L. Griffiths, Sanjit A. Seshia, Pieter Abbeel, and Anca D. Dragan. On the utility of learning about humans for human-ai coordination. *CoRR*, abs/1910.05789, 2019.
- [DJV⁺20] Michael Dennis, Natasha Jaques, Eugene Vinitzky, Alexandre Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. In *Proceedings of the 34th International Conference on Neural Information Processing Systems*, NIPS ’20, Red Hook, NY, USA, 2020. Curran Associates Inc.
- [dWGM⁺20] Christian Schroeder de Witt, Tarun Gupta, Denys Makoviichuk, Viktor Makoviy-chuk, Philip H. S. Torr, Mingfei Sun, and Shimon Whiteson. Is independent learning all you need in the starcraft multi-agent challenge?, 2020.
- [Elm93] Jeffrey L Elman. Learning and development in neural networks: The importance of starting small. *Cognition*, 48(1):71–99, 1993.
- [FFA⁺24] Jakob Foerster, Gregory Farquhar, Triantafyllos Afouras, Nantas Nardelli, and Shimon Whiteson. Counterfactual multi-agent policy gradients, 2024.
- [FHGA18] Carlos Florensa, David Held, Xinyang Geng, and Pieter Abbeel. Automatic goal generation for reinforcement learning agents. In *International conference on machine learning*, pages 1515–1528. PMLR, 2018.
- [GDG⁺24] Samuel Garcin, James Doran, Shangmin Guo, Christopher G. Lucas, and Stefano V. Albrecht. Dred: Zero-shot transfer in reinforcement learning via data-regularised environment design, 2024.
- [GEK17] Jayesh K. Gupta, Maxim Egorov, and Mykel Kochenderfer. Cooperative multi-agent control using deep reinforcement learning. In Gita Sukthankar and Juan A. Rodriguez-Aguilar, editors, *Autonomous Agents and Multiagent Systems*, pages 66–83, Cham, 2017. Springer International Publishing.
- [HLPF21] Hengyuan Hu, Adam Lerer, Alex Peysakhovich, and Jakob Foerster. "other-play" for zero-shot coordination, 2021.
- [JDPH⁺21] Minqi Jiang, Michael Dennis, Jack Parker-Holder, Jakob Foerster, Edward Grefenstette, and Tim Rocktäschel. Replay-guided adversarial environment design. *Advances in Neural Information Processing Systems*, 34:1884–1897, 2021.
- [JGR20] Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. *CoRR*, abs/2010.03934, 2020.
- [LWT⁺17] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. Multi-agent actor-critic for mixed cooperative-competitive environments. *CoRR*, abs/1706.02275, 2017.

- [MLB⁺25] Nathan Monette, Alistair Letcher, Michael Beukman, Matthew T. Jackson, Alexander Rutherford, Alexander D. Goldie, and Jakob N. Foerster. An optimisation framework for unsupervised environment design, 2025.
- [NPL⁺20] Sanmit Narvekar, Bei Peng, Matteo Leonetti, Jivko Sinapov, Matthew E. Taylor, and Peter Stone. Curriculum learning for reinforcement learning domains: A framework and survey. *Journal of Machine Learning Research*, 21(181):1–50, 2020.
- [OMRF25] Bassel Al Omari, Michael Matthews, Alexander Rutherford, and Jakob Nicolaus Foerster. Multi-agent craftax: Benchmarking open-ended multi-agent reinforcement learning at the hyperscale, 2025.
- [OSV08] Frans A Oliehoek, Matthijs TJ Spaan, and Nikos Vlassis. Optimal and approximate q-value functions for decentralized pomdps. *Journal of Artificial Intelligence Research*, 32:289–353, 2008.
- [PCW⁺20] Rémy Portelas, Cédric Colas, Lilian Weng, Katja Hofmann, and Pierre-Yves Oudeyer. Automatic curriculum learning for deep rl: A short survey. In Christian Bessiere, editor, *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20*, pages 4819–4825. International Joint Conferences on Artificial Intelligence Organization, 7 2020. Survey track.
- [PHJD⁺23] Jack Parker-Holder, Minqi Jiang, Michael Dennis, Mikayel Samvelyan, Jakob Foerster, Edward Grefenstette, and Tim Rocktäschel. Evolving curricula with regret-based environment design, 2023.
- [RBPB25] Constantin Ruhdorfer, Matteo Bortoletto, Anna Penzkofer, and Andreas Bulling. The overcooked generalisation challenge: Evaluating cooperation with novel partners in unknown environments using unsupervised environment design, 2025.
- [RBW⁺24] Alexander Rutherford, Michael Beukman, Timon Willi, Bruno Lacerda, Nick Hawes, and Jakob Foerster. No regrets: Investigating and improving regret approximations for curriculum discovery, 2024.
- [RP99] Douglas LT Rohde and David C Plaut. Language acquisition in the absence of explicit negative evidence: How important is starting small? *Cognition*, 72(1):67–109, 1999.
- [RSdW⁺18] Tabish Rashid, Mikayel Samvelyan, Christian Schröder de Witt, Gregory Farquhar, Jakob N. Foerster, and Shimon Whiteson. QMIX: monotonic value function factorisation for deep multi-agent reinforcement learning. *CoRR*, abs/1803.11485, 2018.
- [San94] Terence D Sanger. Neural network learning control of robot manipulators using gradually increasing task difficulty. *IEEE transactions on Robotics and Automation*, 10(3):323–333, 1994.
- [SB18] R.S. Sutton and A.G. Barto. *Reinforcement Learning, second edition: An Introduction*. Adaptive Computation and Machine Learning series. MIT Press, 2018.
- [SKD⁺23] Mikayel Samvelyan, Akbir Khan, Michael Dennis, Minqi Jiang, Jack Parker-Holder, Jakob Foerster, Roberta Raileanu, and Tim Rocktäschel. Maestro: Open-ended environment design for multi-agent reinforcement learning, 2023.
- [SKS⁺19] Guillaume Sartoretti, Justin Kerr, Yunfei Shi, Glenn Wagner, T. K. Satish Kumar, Sven Koenig, and Howie Choset. Primal: Pathfinding via reinforcement and imitation multi-agent learning. *IEEE Robotics and Automation Letters*, 4(3):2378–2385, July 2019.
- [SLG⁺17] Peter Sunehag, Guy Lever, Audrunas Gruslys, Wojciech Marian Czarnecki, Vinícius Flores Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls, and Thore Graepel. Value-decomposition networks for cooperative multi-agent learning. *CoRR*, abs/1706.05296, 2017.
- [SRdW⁺19] Mikayel Samvelyan, Tabish Rashid, Christian Schroeder de Witt, Gregory Farquhar, Nantas Nardelli, Tim G. J. Rudner, Chia-Man Hung, Philip H. S. Torr, Jakob Foerster, and Shimon Whiteson. The starcraft multi-agent challenge, 2019.

- [Tan93] Ming Tan. Multi-agent reinforcement learning: independent versus cooperative agents. In *Proceedings of the Tenth International Conference on International Conference on Machine Learning*, ICML'93, page 330–337, San Francisco, CA, USA, 1993. Morgan Kaufmann Publishers Inc.
- [TFR⁺17] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world, 2017.
- [V⁺11] Lev Vygotsky et al. *Interaction between learning and development*. Linköpings universitet Linköping, Sweden, 2011.